

Une suppression ne remonte jamais au cloud

Analyse ciblée d'un seul défaut - Force Tracker ft-v1234, arbre 442edfbd - 23 septembre 2026.

[!!] **Analyse seulement.** Aucun code modifié, aucune correction appliquée, aucun bump.

Ce dossier ne porte **aucune donnée personnelle** : il décrit un mécanisme de code.

Le défaut en une phrase

Quand quelqu'un **retire** une valeur de son profil, l'application envoie bien la suppression au serveur - mais le serveur la **refuse**, parce qu'il ne sait pas distinguer « cette valeur a été retirée » de « cet appareil ne la connaît pas encore ». [*] L'ancienne valeur reste donc au cloud, et **revient à la première restauration**. [!!] **La suppression n'est pas perdue en transit : elle est refusée, en silence, sans trace.**

1. Le mécanisme - quatre fusionneurs, exécutés et non décrits

Le backend porte quatre garde-fous, ajoutés sous le titre « *le vide ne gagne jamais sur du rempli* » (Code.js:1535-1543). Ils protègent **50 champs** du profil. [*] Ce document ne les décrit pas : il les **extraît de Code.js et les exécute** à chaque génération.

Les fusionneurs relus dans le code servi, puis EXECUTES par node à la génération de ce PDF :

```
function _pn_(b, e){ if(b===undefined)return e; return (b&&b!==0)?b:(e||b||0); }
function _ps_(b, e){ if(b===undefined)return e; return (b&&b!=='')?b:(e||b||''); }

_pn_(0, 2500)      -> 2500      le 0 envoyé est IGNORE, l'ancien reste
_pn_(2500, 0)      -> 2500      une vraie valeur, elle, passe
_pn_(0, 0)         -> 0         rien à protéger, le 0 passe
_pn_(undefined, 90) -> 90       champ absent : ignore (voulu)
_ps_(' ', 'perte') -> "perte"   la chaîne vide est IGNOREE
_pa_([], ['force']) -> ["force"] le tableau vide est IGNORE
_po_({}, {sq:100})  -> {"sq":100} l'objet vide est IGNORE
```

[!!] **Le garde-fou ne distingue pas « absente » de « retirée ».** Les deux arrivent au serveur sous la même forme : 0, ' ', [] ou {}.

2. La chaîne complète, ancre par ancre

Etape	Ce qui se passe	Ancre
Suppression locale	S.manualKcal = 0	screens.js:2540
Ce qui part au cloud	manualKcal: S.manualKcal 0 -> 0	setup.js:922
Ce que le backend en fait	_pn_(0, 2500) -> 2500	Code.js:1669
Ce qui reste au cloud	L'ancienne valeur, intacte	Script Properties
A la restauration	if(d.manualKcal) : 2500 est <i>truthy</i> -> elle revient	setup.js:3341
Au prochain enregistrement	la valeur ressuscitée repart au cloud : la suppression est définitivement perdue	setup.js:922

3. Données concernées - trois chemins prouvés, pas un de plus

Le **mécanisme** couvre les 50 champs protégés. Mais une suppression n'est un problème que s'il existe un chemin pour l'effectuer. **Trois sont prouvés dans le code servi** :

Champ	Action	Où	Forme de la suppression
manualKcal	bouton « remettre en automatique » toast « Calories remises en automatique »	screens.js:2540	0
targetWeight	champ vide -> toast « Objectif retiré »	tracking.js:753	0
goal2	retrait automatique quand l'objectif devient incompatible - pas une action de la personne	setup.js:2727	' '

[!!] **Pas de généralisation au reste du profil.** Pour les 47 autres champs protégés, aucune action utilisateur qui les vide n'a été trouvée. *Absence de preuve, pas preuve d'absence.*

4. Scenario utilisateur

AVANT	cible manuelle 2 500 kcal, posee il y a des mois locale = 2 500 cloud = 2 500
ACTION	Nutrition -> modifier la cible -> « Remettre en automatique » toast : « Calories remises en automatique »
ETAT LOCAL	S.manualKcal = 0 -> la cible redevient calculee [ok]
ETAT CLOUD	envoie 0 -> _pn_(0, 2500) -> reste 2 500 [X]
RESTAURATION	changement de telephone, ou navigateur vide _applyRestoreData : if(d.manualKcal) -> 2 500 revient
RESULTAT	la cible calorique est de nouveau figee a 2 500 kcal. Aucun message. Et le prochain enregistrement renvoie 2 500 au cloud : la suppression est perdue pour de bon.

Frequence reelle - ca ne mord pas tous les jours

_applyRestoreData est appele **4 fois** dans app.js, et l'auto-restauration silencieuse est **bornee par _localEmpty** : seances **et** records **et** programmes tous vides. [*] Le default se declenche donc a l'**inscription**, a une **restauration explicite**, ou apres une **purge locale totale** - pas a chaque demarrage.

5. Cause racine

Le protocole de synchronisation **n a pas de representation de la suppression**. Un champ retire et un champ jamais renseigne arrivent au serveur sous la meme forme. Le garde-fou - ecrit pour une **bonne raison** : empecher un appareil non restaure d'ecraser un profil rempli - tranche donc systematiquement en faveur de l'ancienne valeur.

Ce qu'on a cherche	Resultat mesure
Une route d'effacement cote serveur	Aucune . Les 34 actions reconnues sont : adminRestore, aiCount, aiUsageLog, authIdentity, authStatus, bodyStudy, coach, estimateFood, foodLabel, generateMealPlan, importBloodTest, importBodyScan, importHistory, importMealPlan, importProgram, issueTokenByCode, listUsers, loadProfile, logCustomExercise, logSearchMiss, logSession, migStats, morphoAnalysis, pushHealth, readBarcode, revokeToken, saveProfile, sendConfirmCode, setAccessCode, summarizeCoach, test, testerIdea, validateCode, verifyConfirmCode
Un tombstone, un marqueur de suppression	Aucun - le mot n'apparait pas dans Code.js
Une valeur speciale reconnue comme « retire »	Aucune
Un horodatage pour arbitrer les conflits	Aucun
Une difference GET / POST sur ce point	Aucune : les deux reponses rendent profile: data.profile {} a l'identique (2 occurrences)

6. Rayon reel et severite

Classement	Avant	Apres analyse	Pourquoi
Rayon	SYSTEMIQUE	MULTI-MOTEUR	manualKcal pilote la cible calorique, les trois macros et le contexte de Milo - donc plusieurs moteurs. Mais trois champs seulement sont atteignables, sur un seul chemin . Le reste du profil n'a pas de chemin de suppression prouve.
Severite	MAJEUR	MAJEUR (confirme)	Malgre la faible frequence : c'est silencieux (aucun message, aucune trace serveur), irreversible (le prochain enregistrement renvoie la valeur ressuscitee), et ca survient quand la personne n'a aucun repere - sur un telephone neuf.

7. Correction envisageable PLUS TARD - rien n'est applique

Piste	Cout	Ce qu'elle change
1. Un marqueur de suppression : envoyer null (et non 0) pour un retrait explicite, et faire de null le seul cas reconnu comme « retire » par les quatre fusionneurs. undefined reste « je ne sais pas ».	faible	ne touche pas le garde-fou d'origine
2. Une liste <code>_supprimes</code> : ['manualKcal'] dans le corps de <code>saveProfile</code> , traitée avant les fusionneurs.	moyen	ajoute un champ au protocole
3. Un horodatage par champ - la vraie réponse au problème général.	élevé	change le format de stockage

La contrainte a ne pas perdre de vue

Le garde-fou existe parce qu'un appareil **non restaure** envoyait des champs vides et **écrasait un profil rempli**. [*] **Toute correction doit garder ce cas fermé**. Retirer le garde-fou pour laisser passer les suppressions rouvrirait le défaut qu'il a été écrit pour fermer.

8. Points encore INCONNUS

Point	Pourquoi il reste inconnu
Le comportement du backend deploye	Code.js est lu dans le dépôt, il n'est pas exécuté ici : le proxy de ce conteneur n'autorise que GitHub. Le raisonnement repose sur le code source , pas sur une réponse réelle du serveur.
La valeur réellement stockée pour un compte donne	Les Script Properties ne sont pas observables d'ici.
Le miroir Supabase	Il reçoit le blob entier (setup.js : 910-915) et n'a pas été audité : il pourrait se comporter autrement.
Les 47 autres champs protégés	Le mécanisme les couvre, mais aucun chemin de suppression n'a été prouvé pour eux.

9. Points connexes - notes, PAS suivis

Point connexe à vérifier plus tard	Ancre
healthProfile est traité par <code>body.healthProfile profile.healthProfile null</code> , hors des quatre fusionneurs - même motif, sur une donnée de santé.	Code.js:1685
La réponse GET de <code>loadProfile</code> ne renvoie ni programmes, ni <code>exRestPref</code> , ni <code>exSwaps</code> , la ou la POST les renvoie. Sans rapport avec la suppression, mais les deux routes ont divergé.	Code.js:1027 vs 1068

Etat du dépôt, vérifié par git à la génération

AUCUN FICHIER SERVI N'EST MODIFIÉ (state.js, app.js, screens.js, coach.js, setup.js, tracking.js, sw.js, Code.js). Aucun bump - le cache sert toujours **ft-v1234**. Aucune publication. [*] *Ce dossier explique un mécanisme ; il ne corrige rien.*

Généré par `tools/gen_suppression_cloud_pdf.py`. Ses **34 gardes** relisent chaque ancre dans le code servi et **refusent de produire** si un seul fait tombe. [!!] Le garde central n'est pas un recomptage : les quatre fusionneurs sont **extraits de Code.js et exécutés** - si `_pn_(0, 2500)` cessait de rendre 2500, ce document ne sortirait pas.